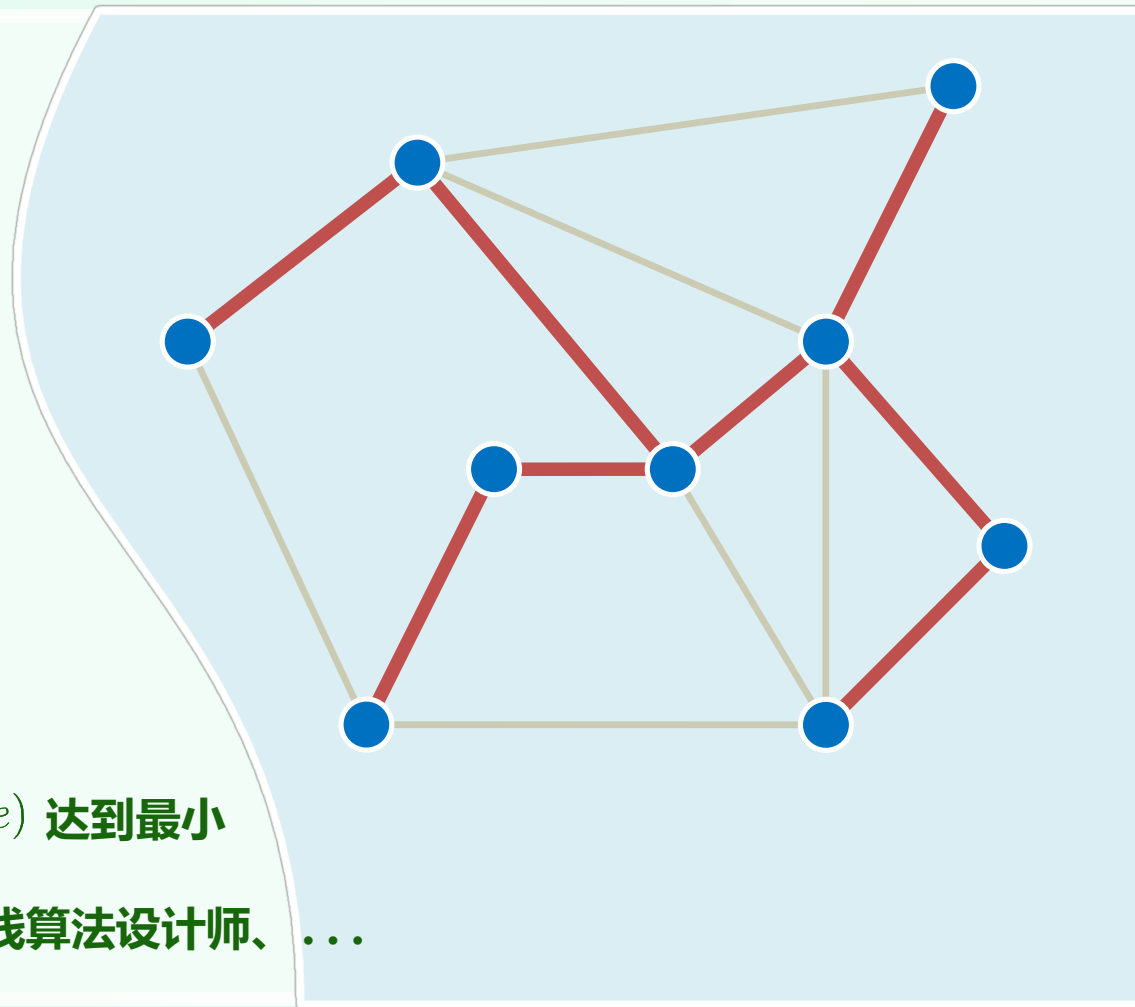


图应用

Prim算法： 最小支撑树

最小 + 支撑 + 树

- ❖ 连通网络 $N = (V; E)$ 的子图 $T = (V; F)$
- ❖ 支撑/spanning = 覆盖 N 中所有顶点
- ❖ 树/tree =
 - 连通且无环, $|V| = |F| + 1$
 - 加边出单环, 再删同环边即恢复为树
 - 删边不连通, 再加联接边即恢复为树
- ❖ 不难验证, 同一网络的支撑树不唯一
- ❖ 最小/minimum: 各边总权重 $w(T) = \sum_{e \in F} w(e)$ 达到最小
- ❖ 谁感兴趣? 电信公司、网络设计师、VLSI布线算法设计师、...



❖ 为何重要?

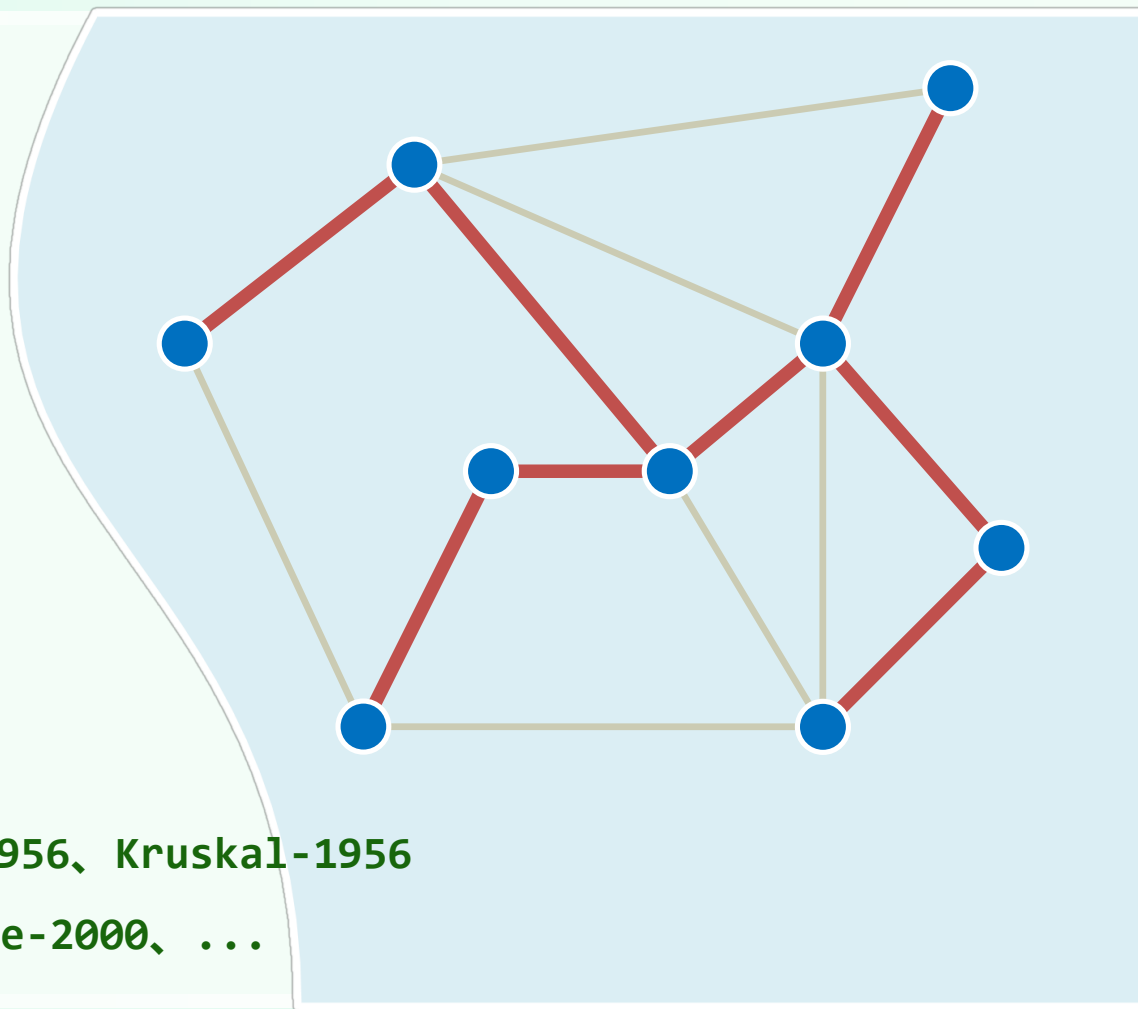
- 自身可**有效计算**
 - 众多优化问题的**基本模型**
 - 为许多NP问题提供足够好的**近似解**
- 比如, Euclidean TSP

✦ 延伸问题

- Proximity Graphs // $\Omega(n \log n)$
- Steiner MST // NP-hard

❖ 已有算法

- Boruvka-1926、Jarnik-1930、Prim-1956、Kruskal-1956
Karger-Klein-Tarjan-1995、Chazelle-2000、...



退化

❖ 权值必须是正数?

- 允许为零, 有何影响?
- 允许为负数呢?

❖ 所有支撑树所含的边数, 必相等

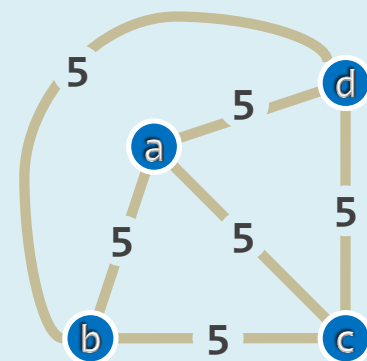
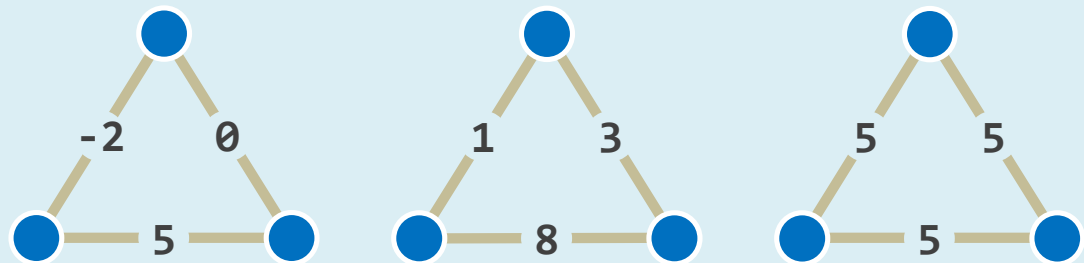
故可统一调整: `increase(1 - findMin())`

❖ The minimum?

- A minimal: 同一网络或有多棵MST
- 可强制消除歧义...

❖ 合成数 (composite number) : $(w(u, v), \min(u, v), \max(u, v))$

$5ab < 5ac < 5ad < 5bc < 5bd < 5cd$



蛮力算法

❖ 枚举出N的所有支撑树，从中找出代价最小者

❖ 包含n个顶点的图，可能有多少棵支撑树？

n = 1 1

n = 2 1

n = 3 3

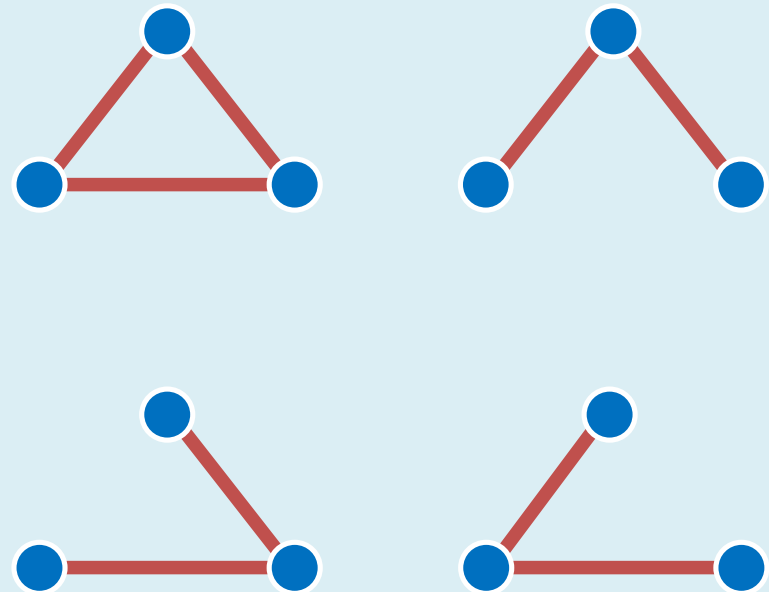
n = 4 16

n = 5 125

... ...

❖ Cayley公式：完全图 K_n 有 n^{n-2} 棵支撑树

❖ 如何高效地构造MST呢？

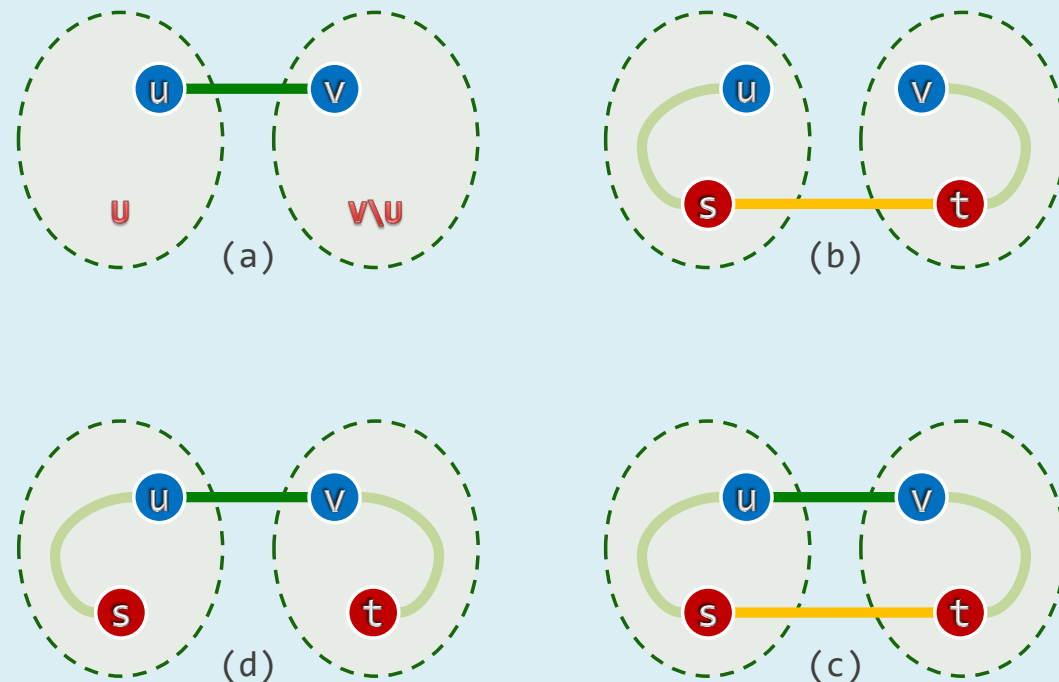


图应用

Prim算法： 极短跨边

割 & 极短跨边

- ❖ 设 $(U; V \setminus U)$ 是 N 的一个割 (Cut)
- ❖ 若 uv 是该割的一条**极短跨边**
则必**存在**一棵包含 uv 的MST
- ❖ 反证：假设 uv 未被**任何**MST采用...
任取一棵MST，将 uv 加入其中，于是
 - 将出现**唯一**的回路，且该回路
 - 必经过 uv 以及**至少**另一跨边 st接下来，摘除 st 后...
恢复为一棵支撑树，且总权重不致增加
- ❖ 反之，**任一**MST都必然通过极短跨边联接**每一**割



递增式构造

❖ 首先, 任选: $T_1 = (\{v_1\}; \emptyset)$

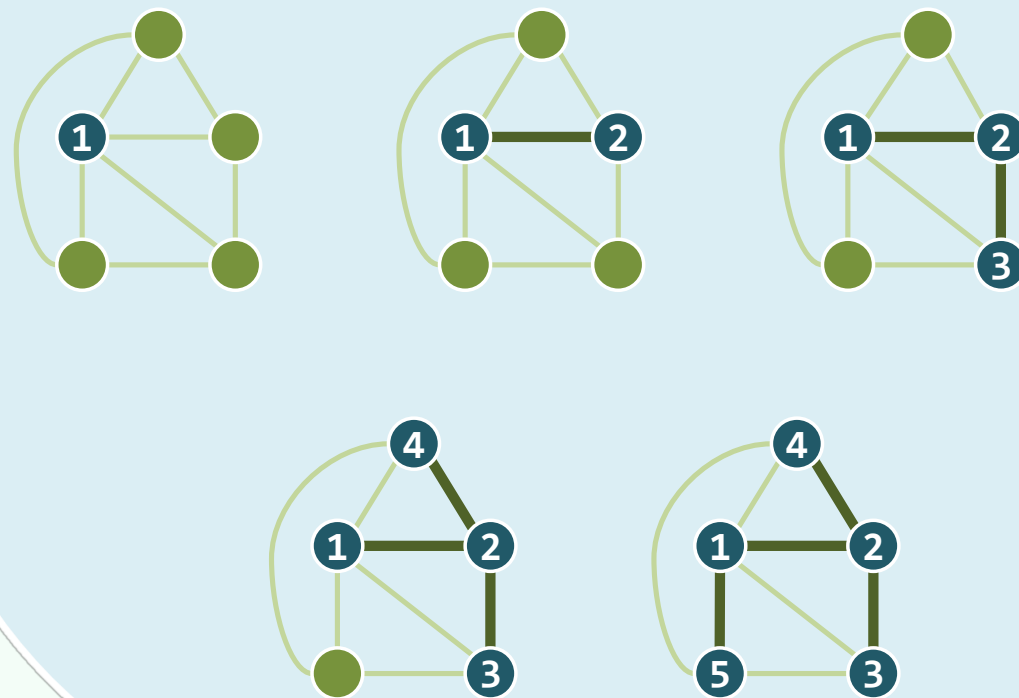
❖ 以下, 不断地将 T_k 拓展为树 T_{k+1}

$$\begin{aligned} T_{k+1} &= (V_{k+1}; E_{k+1}) \\ &= (V_k \cup \{v_{k+1}\}; E_k \cup \{v_{k+1}u\}) \end{aligned}$$

其中, $u \in V_k$

❖ 由此前的分析

- 只需将 $(V_k; V \setminus V_k)$ 视作原图的一个割
- 该割所有跨边中的**极短者**即是 $v_{k+1}u$



图应用

Prim算法： 正确性

似是而非

❖ 设Prim算法依次选取了边 $\{ e_2, e_3, \dots, e_n \}$

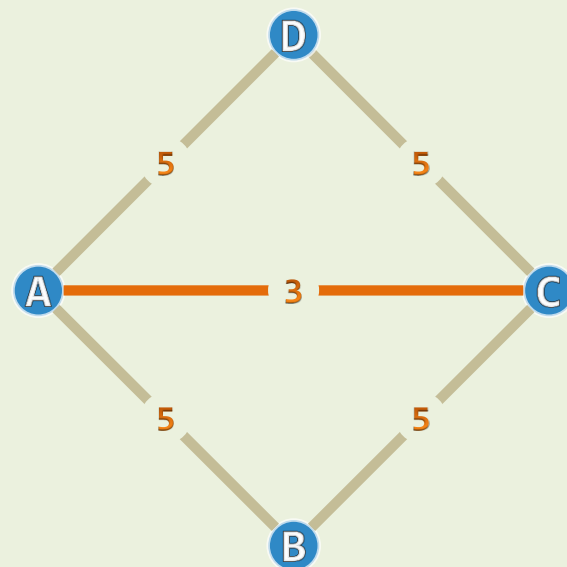
❖ 其中每一条边 e_k , 的确都属于**某棵**MST

❖ 但在MST**不唯一**时...

由此并不能确认, 最终的T必是MST (之一)

❖ 由极短跨边构成的支撑树, 未必就是一棵MST

反例...

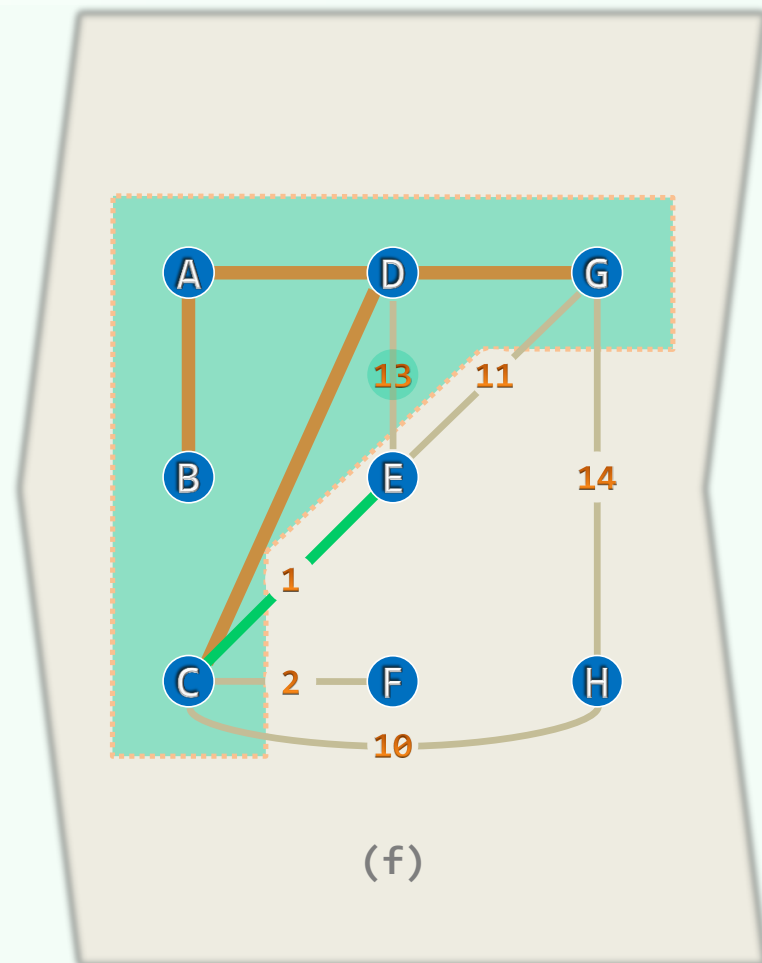


图应用

Prim算法： 实现

PFS

- ❖ $\forall v \notin V_k$, let $\text{priority}(v) = \|s, v\| \leq \infty$
- ❖ 于是套用PFS框架, 为将 T_k 扩充至 T_{k+1} , 只需
 - 选出优先级最高的跨边 e_k 及其对应顶点 u_k , 并将其加入 T_k
 - 随后, 更新 $V \setminus V_{k+1}$ 中所有顶点的优先级 (数)
- ❖ 注意: 优先级数随后可能改变 (降低) 的顶点, 必与 u_k 邻接
- ❖ 因此, 只需枚举 u_k 的每一邻接顶点 v , 并取
$$\text{priority}(v) = \min(\text{priority}(v), \|u_k, v\|)$$
- ❖ 以上完全符合PFS的框架, 唯一要做的工作无非是
按照prioUpdater()规范, 编写一个优先级 (数) 更新器...



PrioUpdater()

```
g->pfs( 0, PrimPU<char, Rank>() ); //从顶点0出发, 启动Prim算法

template <typename Tv, typename Te> struct PrimPU { //Prim算法的顶点优先级更新器

    virtual void operator()( Graph<Tv, Te>* g, Rank s, Rank w ) { //对s的每个

        if ( UNDISCOVERED != g->status(w) ) return; //尚未被发现的邻居w, 按

        if ( g->priority(w) > g->weight(s, w) ) { //Prim

            g->priority(w) = g->weight(s, w); //策略

            g->parent(w)    = s; //做松弛

        }

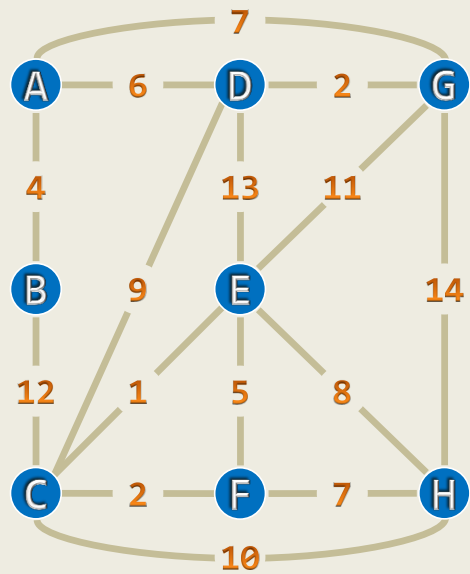
    }

};
```

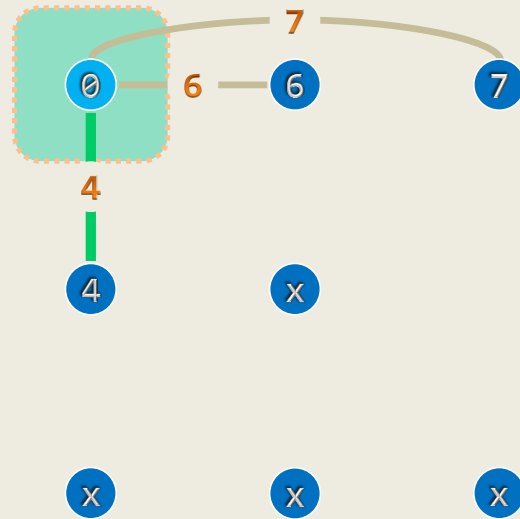
图应用

Prim算法： 实例

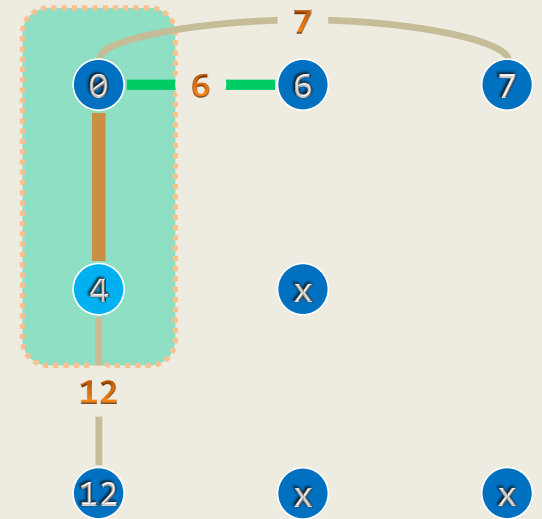
1/3



(a)

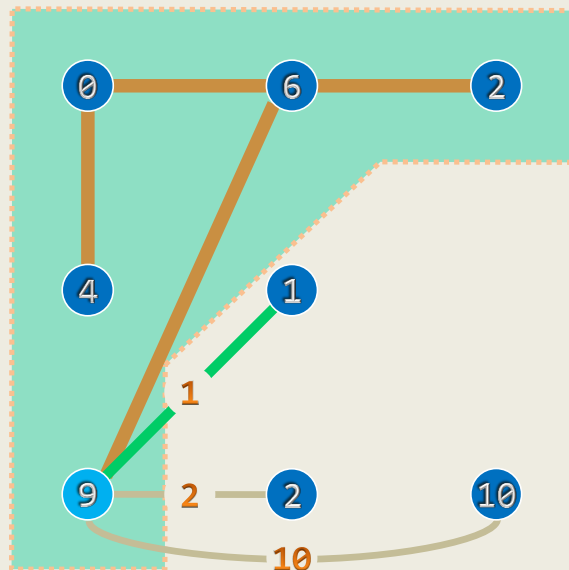


(b)

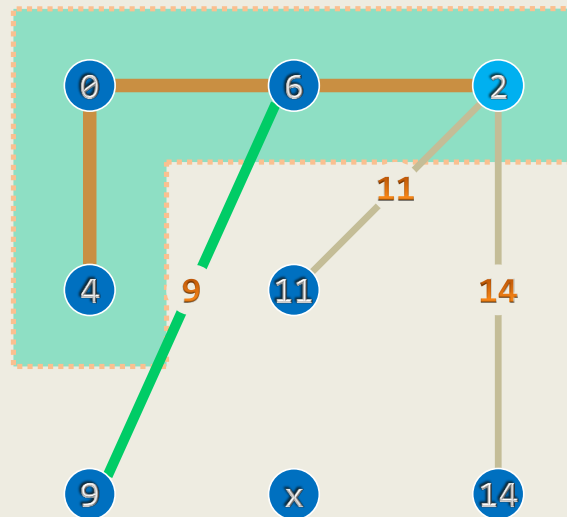


(c)

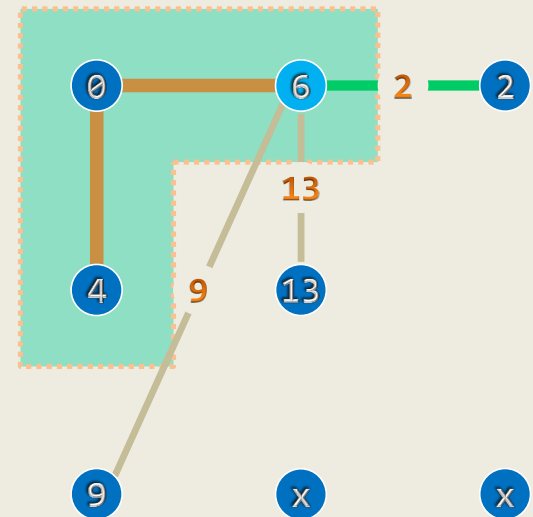
2/3



(f)

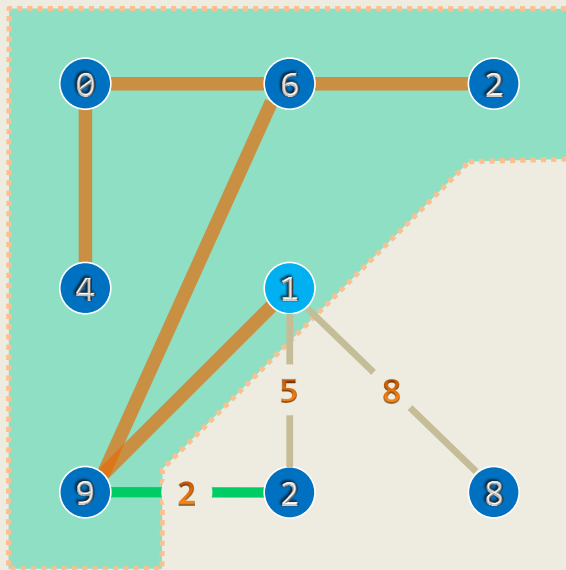


(e)

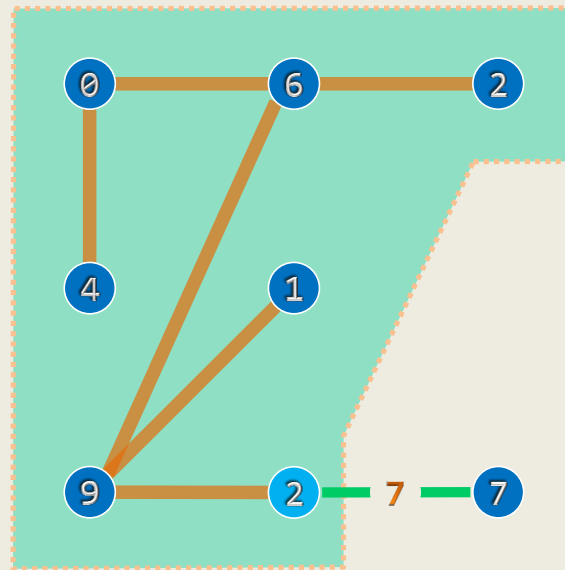


(d)

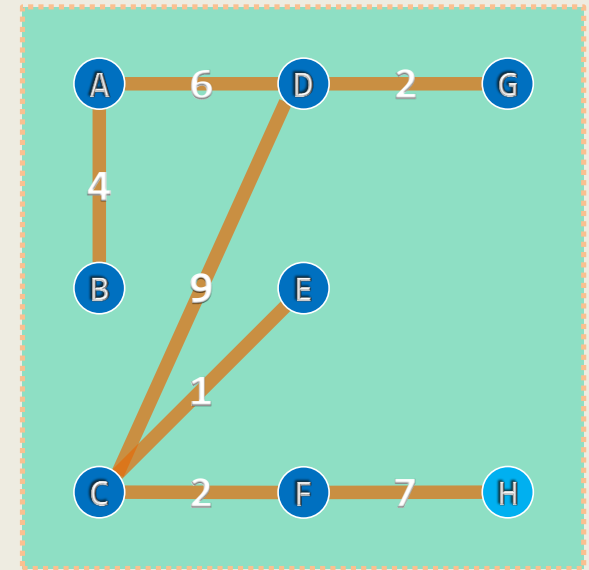
3/3



(g)



(h)



(i)

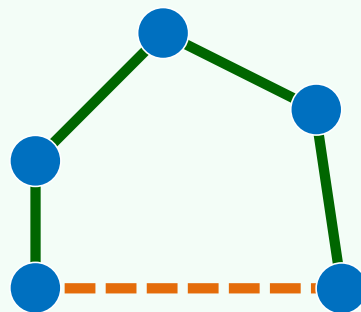
图应用

Kruskal算法： 算法

贪心策略

❖ 回顾Prim算法

- 最短边，迟早会被采用
- 次短边，亦是如此
- 再次短者，则未必 //回路!

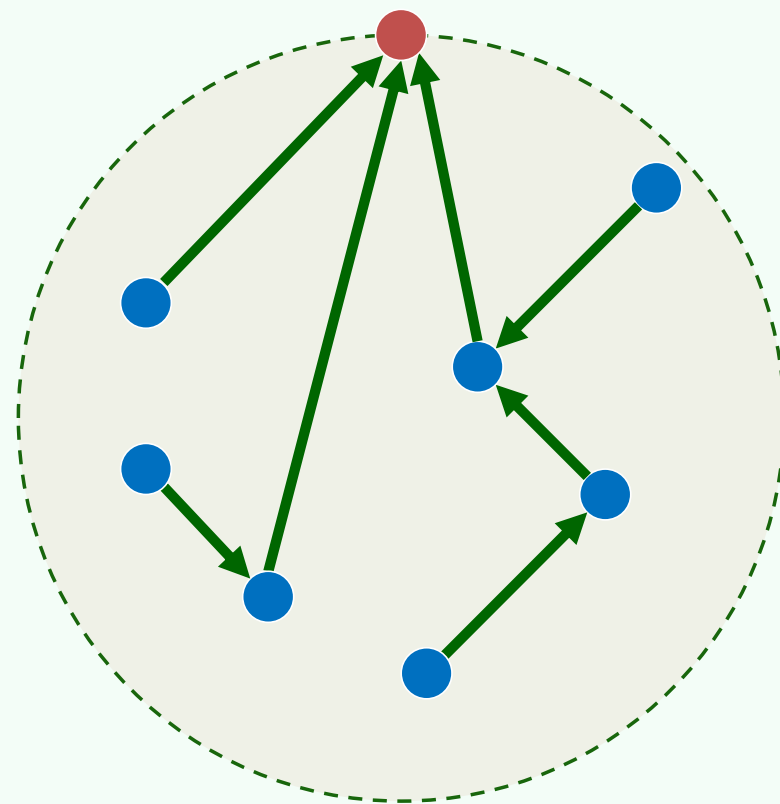


❖ Kruskal: 贪心原则

- 根据代价，从小到大依次尝试各边
- 只要“安全”，就加入该边

❖ 但是，每步局部最优 = 全局最优?

❖ 确实，Kruskal很幸运...



正确性

❖ 定理: Kruskal引入的每条边都属于**某棵**MST

❖ 设: 边 $e = (u, v)$ 的引入导致树 T 和 S 的合并

❖ 若: 将 $(T; V \setminus T)$ 视作原网络 N 的割

则: e 当属该割的一条**跨边**

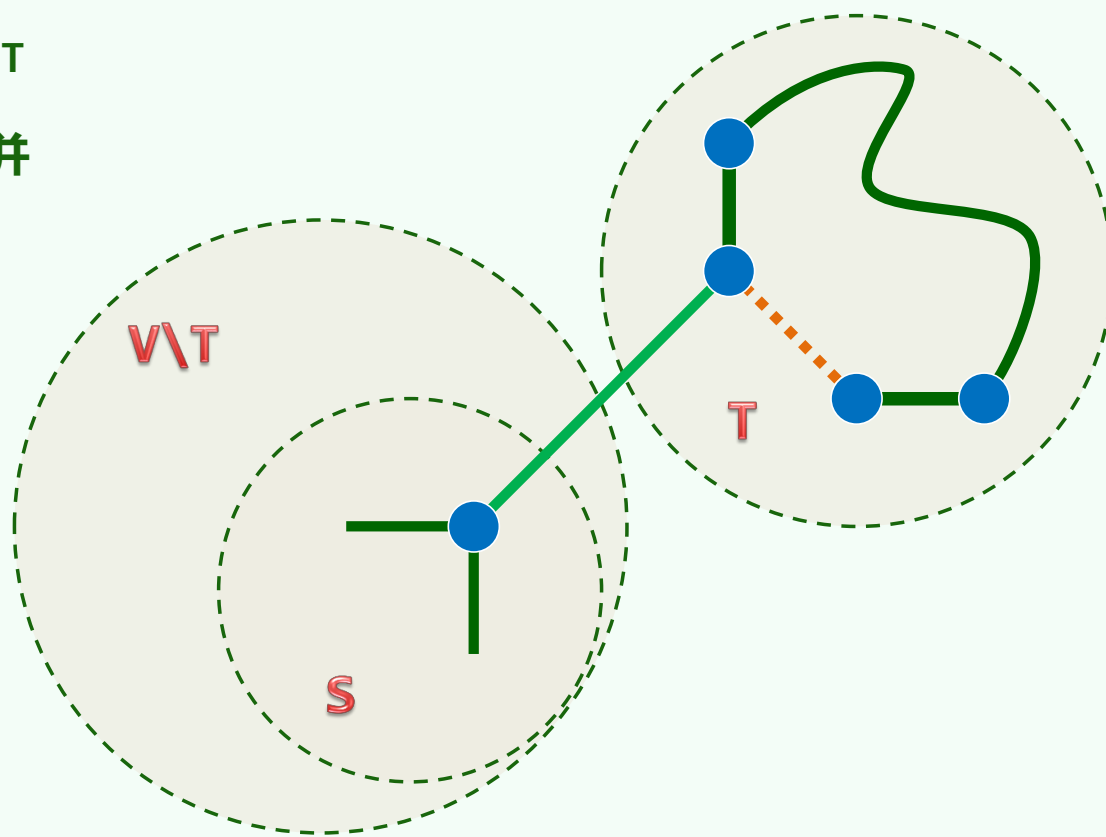
❖ 在确定应引入 e 之前

- 该割的所有跨边都经Kruskal考察
- 且只可能因不短于 e 而被淘汰

❖ 故: e 当属该割的一条**极短跨边**

❖ 与Prim同理, 以上论述也不充分

为严格起见, 仍需归纳证明: Kruskal算法过程中不断生长的森林, 总是**某棵**MST的**子图**



算法

❖ 维护 N 的一个森林: $F = (V; E') \subseteq N = (V; E)$

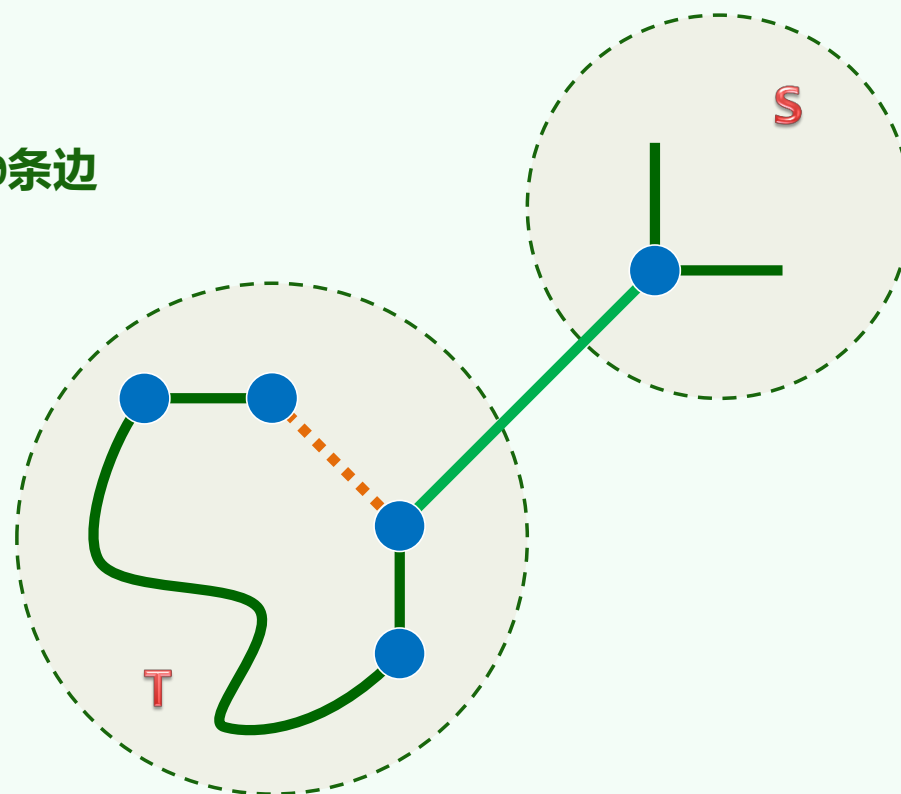
❖ 初始化

- $F = (V; \emptyset)$ 包含 n 棵树 (各含 1 个顶点) 和 0 条边
- 将所有边按照代价排序

❖ 迭代, 直到 F 成为1棵树

- 找到当前最廉价的边 e
- 若 e 的顶点来自 F 中不同的树, 则
 - 令 $E' = E' \cup \{e\}$, 然后
 - 将 e 联接的2棵树合二为一

❖ 整个过程共迭代 $n-1$ 次, 选出 $n-1$ 条边



图应用

Kruskal算法：并查集

Union-Find

❖ Union-Find问题

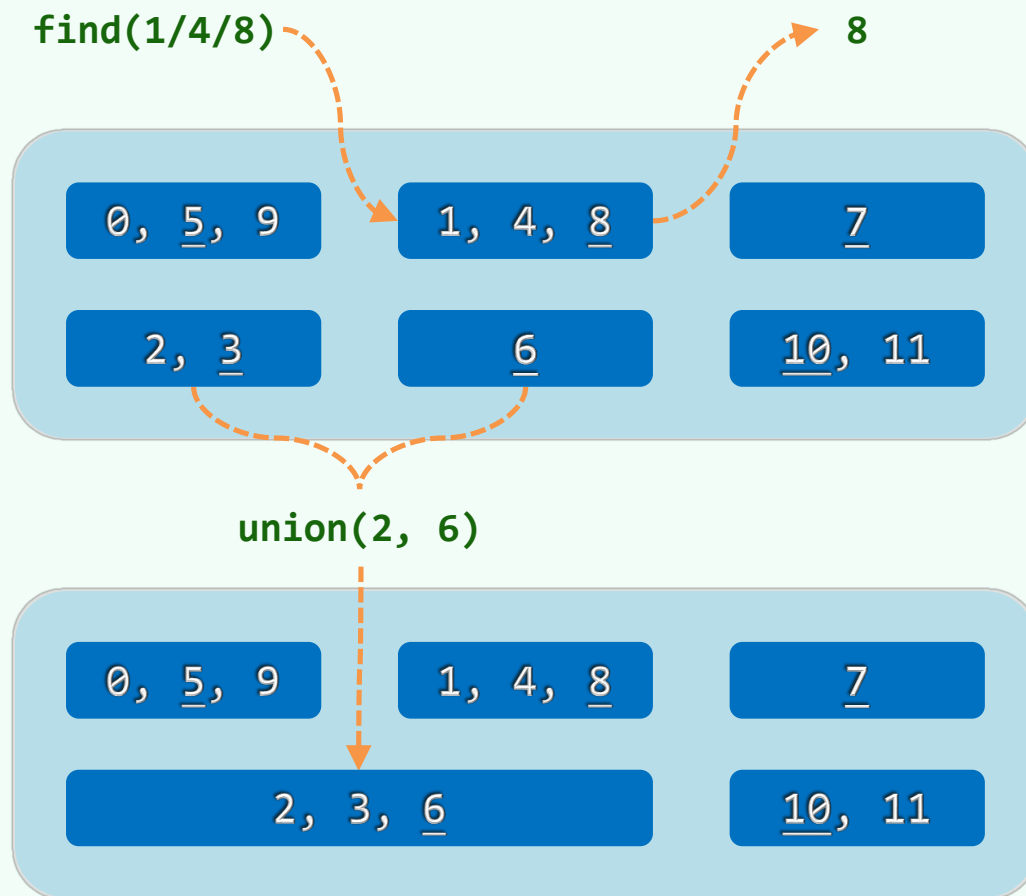
- 给定一组互不相交的等价类
- 各由其中一个成员作为代表

❖ Find(x): 找到元素x所属等价类

❖ Union(x, y): 合并x和y所属等价类

❖ Singleton: 初始时各包含一个元素

❖ Kruskal = Union-Find



Quick-Find

❖ class UnionFind:

```
def __init__(self, n):
```

```
    self.g = self.n = n; self.group = [ k for k in range(n) ]
```

#group[]记录各元素所属子集; 初始各成一类, 就以[0, n)之间整数标识

```
def find(self, k):
```

```
    return self.group[k]
```

0	1	2	3	4	5	6	7	8	9	10	11
0	1	2	3	4	5	6	7	8	9	10	11

```
def union(self, i, j):
```

```
    iGroup , jGroup = self.group[i] , self.group[j]
```

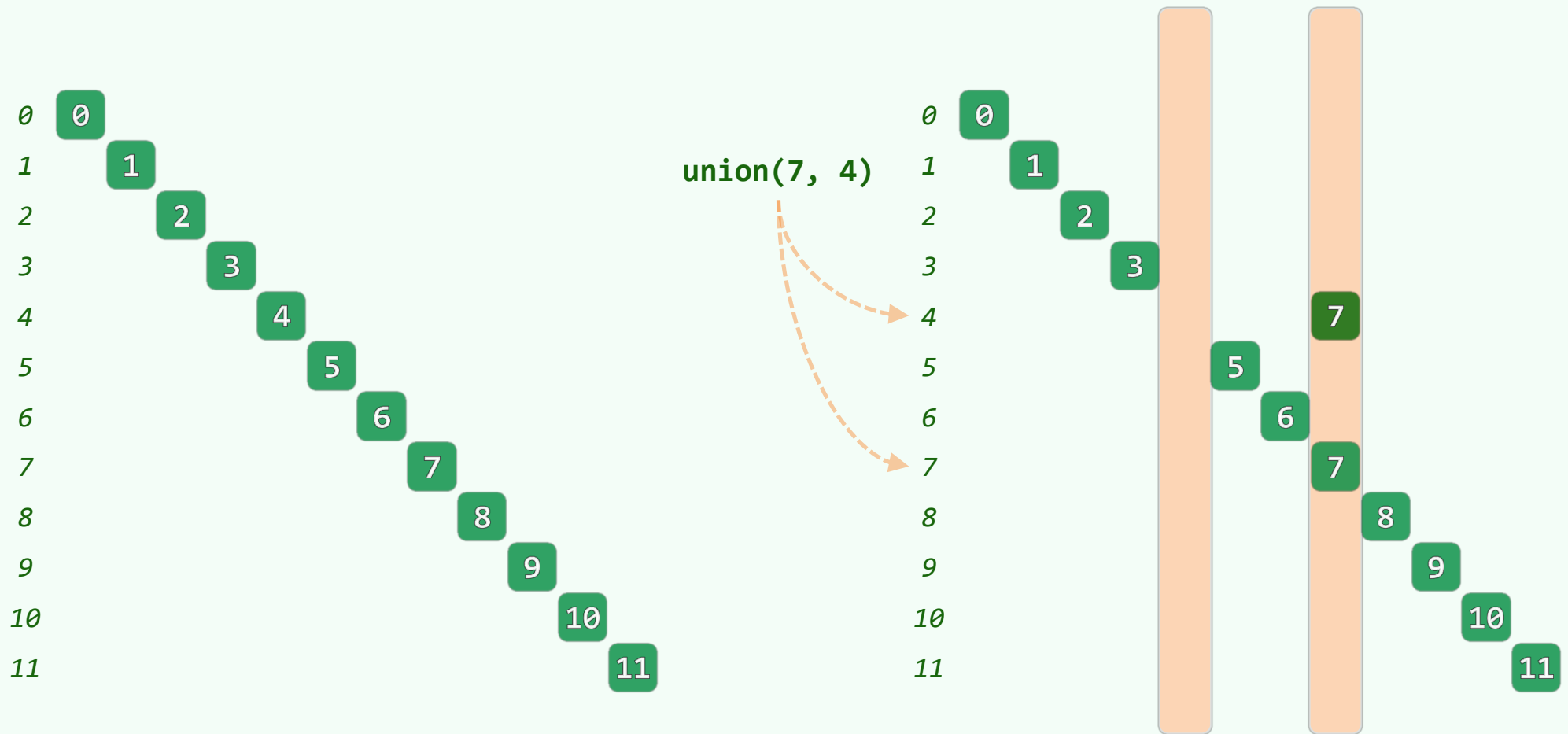
```
    if iGroup == jGroup: return
```

```
    for k in range(self.n):
```

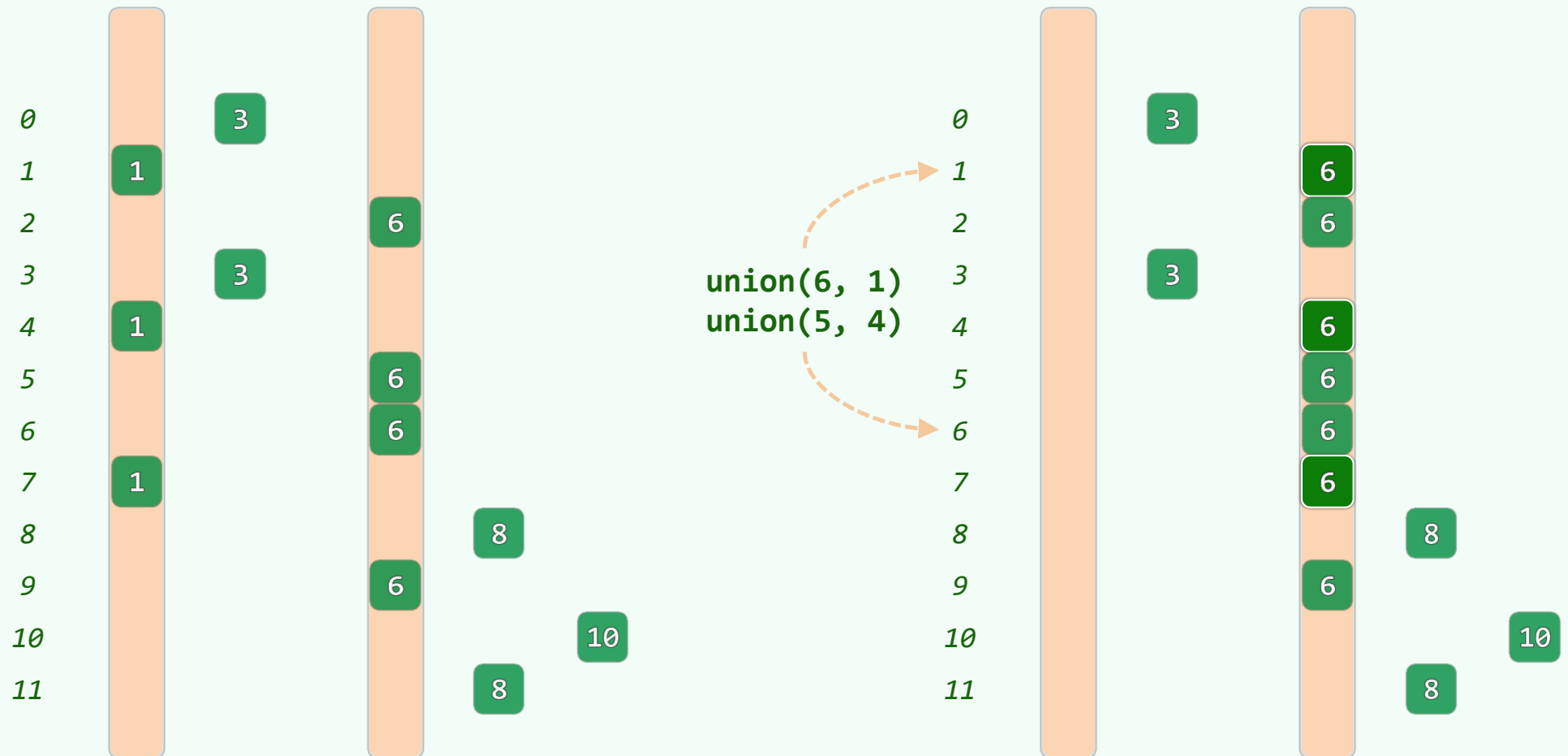
```
        if (self.group[k] == jGroup): self.group[k] = iGroup
```

```
    self.g -= 1
```

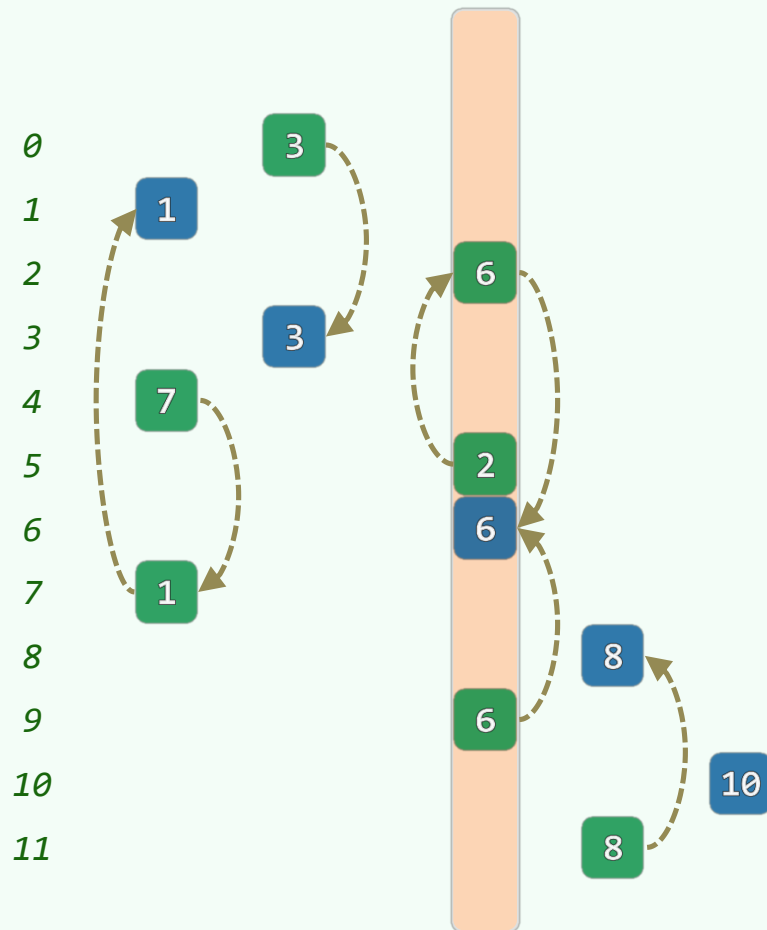

Quick-Find



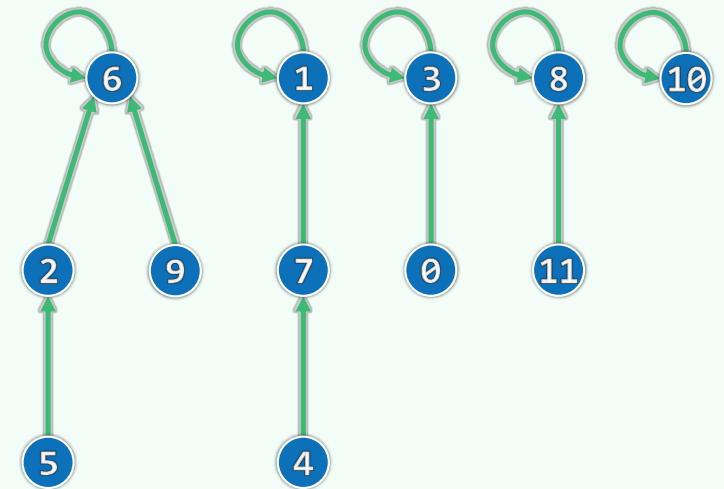
Slow-Union



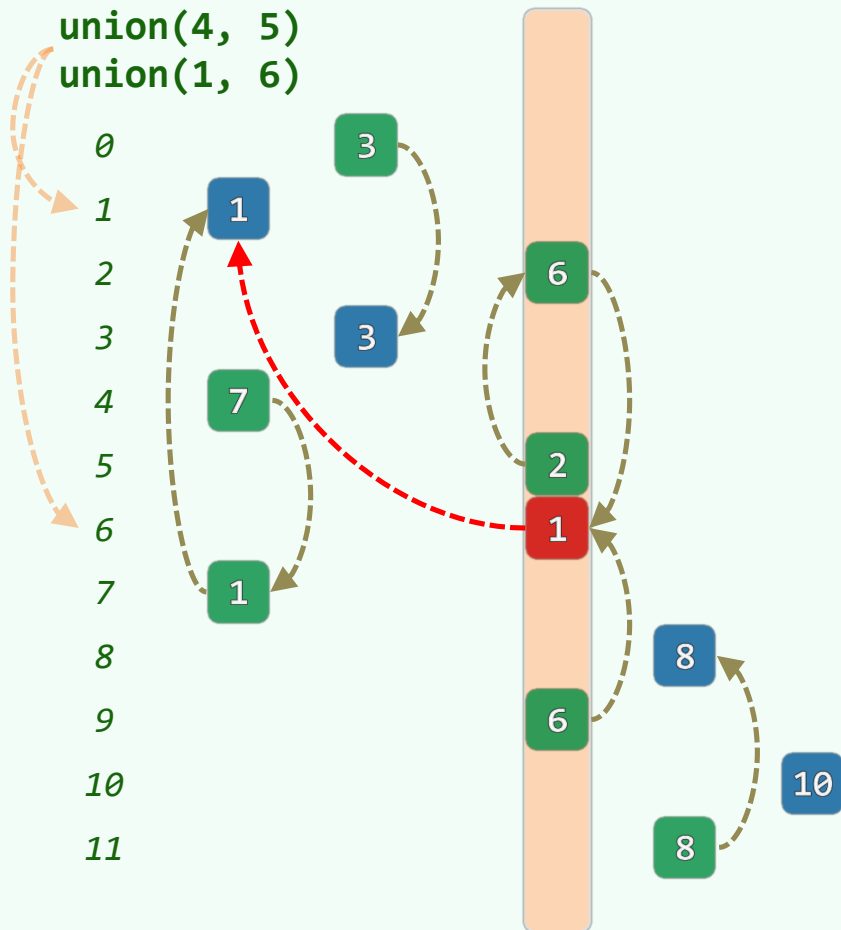
Group ~ Parent



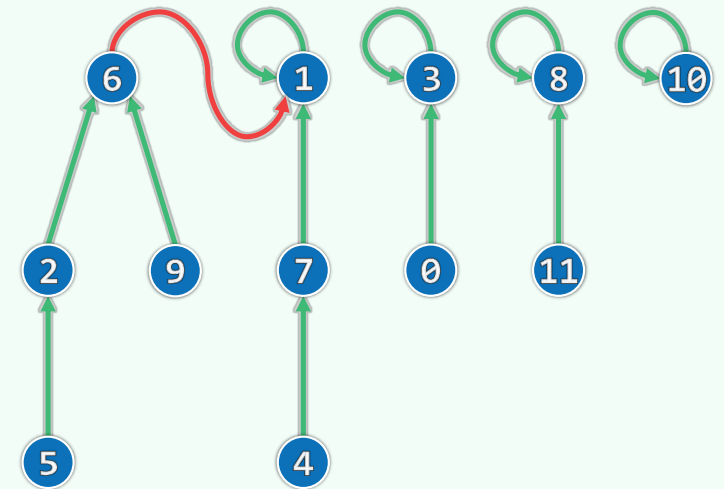
rank	parent
0	3
1	1
2	6
3	3
4	7
5	2
6	6
7	1
8	8
9	6
10	10
11	8



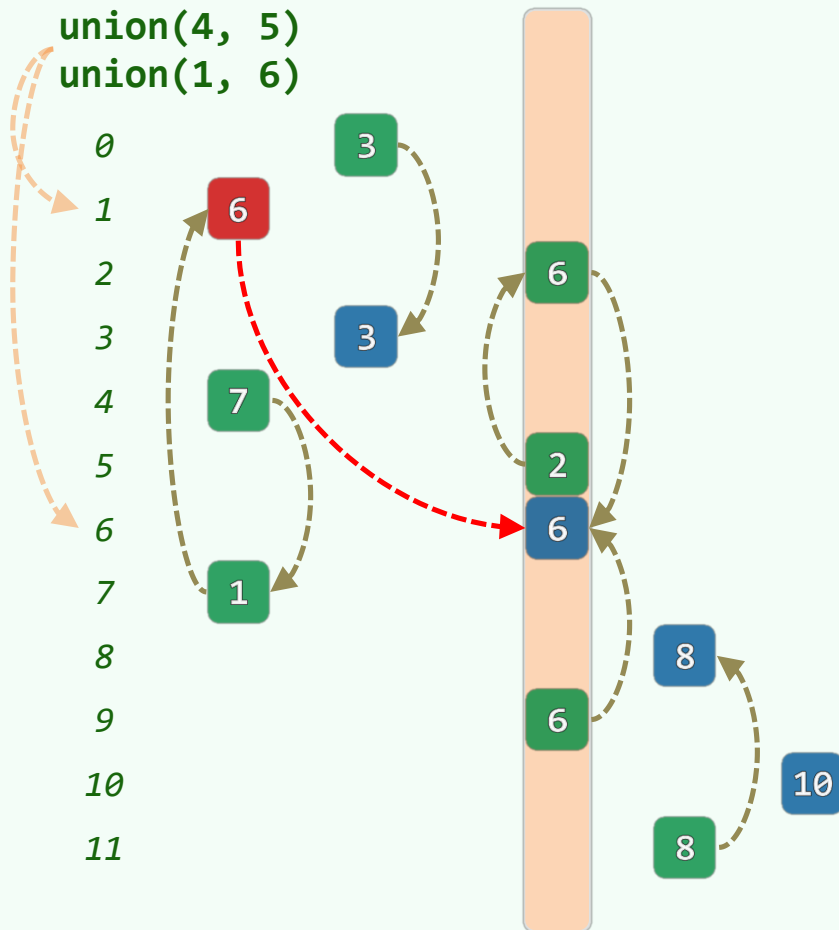
Quick-Union



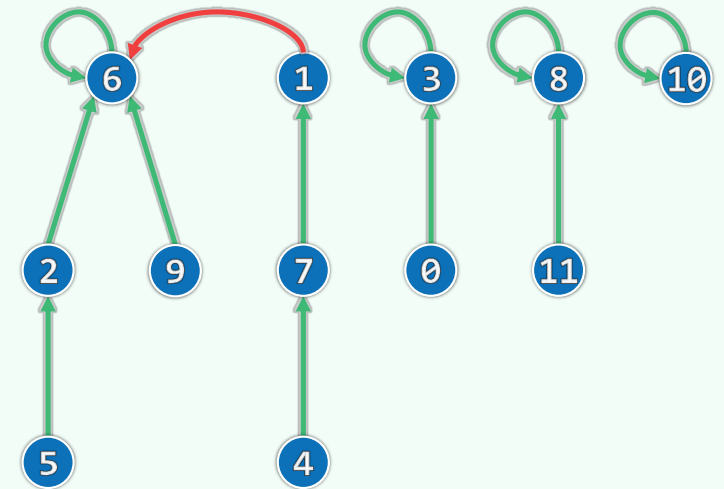
rank	parent
0	3
1	1
2	6
3	3
4	7
5	2
6	1
7	1
8	8
9	6
10	10
11	8



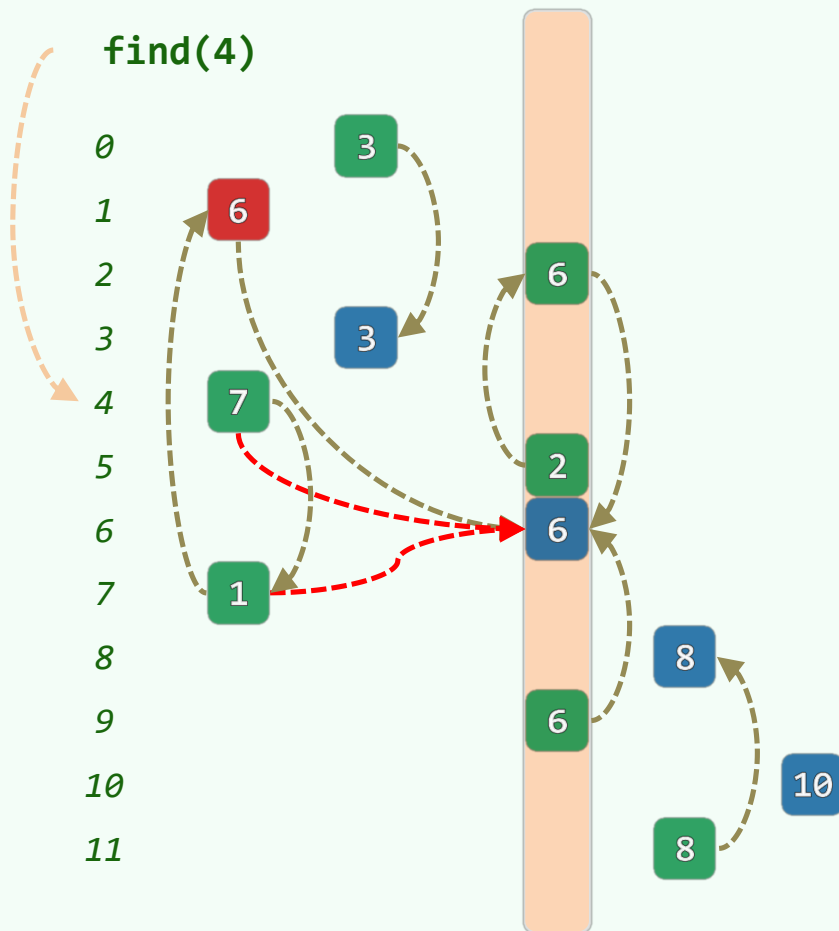
Weighting



rank	parent	size
0	3	
1	6	3
2	6	
3	3	2
4	7	
5	2	
6	6	47
7	1	
8	8	2
9	6	
10	10	1
11	8	



Path Compression



rank	parent	size
0	3	
1	6	
2	6	
3	3	2
4	6	
5	2	
6	6	7
7	6	
8	8	2
9	6	
10	10	1
11	8	

